# OS ORANGE

NAME : ANIRUDHA ALAGAWADI

PES2UG24CS067

## PHASE 1:

### 1A

```
oper@oper-virtualbox:~/PES2UG24CS067-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:138:1: warning: ignoring return value of ‘write’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  138 | write(fd, obj, obj_len);
      | ^~~~~~~~~~~~~~~~~~~~~~~
object.c: In function ‘object_read’:
object.c:194:1: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  194 | fread(raw, 1, size, f);
      | ^~~~~~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

### 1B

```
oper@oper-virtualbox:~/PES2UG24CS067-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

# PHASE 2:

## 2A:

```
oper@oper-virtualbox:~/PES2UG24CS067-pes-vcs$ gcc -Wall -Wextra -O2 test_tree.c tree.c object.c index.c -lcrypto -o test_tree
./test_tree
tree.c: In function 'write_tree_recursive':
tree.c:198:14: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
  198 |     int rc = object_write(OBJ_TREE, data, data_len, id_out);
      |              ^~~~~~~~~~~~
tree.c:185:13: warning: '__builtin___strncpy_chk' output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
  185 |             strncpy(e->name, dir_name, sizeof(e->name)-1);
      |             ^
object.c: In function 'object_write':
object.c:138:1: warning: ignoring return value of 'write' declared with attribute 'warn_unused_result' [-Wunused-result]
  138 | write(fd, obj, obj_len);
      | ^~~~~~~~~~~~~~~~~~~~~~~
object.c: In function 'object_read':
object.c:194:1: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  194 | fread(raw, 1, size, f);
      | ^~~~~~~~~~~~~~~~~~~~~~
index.c:220:1: warning: data definition has no type or storage class
  220 | index_load(index);
      | ^~~~~~~~~~
index.c:220:1: warning: type defaults to 'int' in declaration of 'index_load' [-Wimplicit-int]
index.c:220:1: warning: parameter names (without types) in function declaration
index.c: In function 'index_add':
index.c:247:9: warning: implicit declaration of function 'object_write' [-Wimplicit-function-declaration]
  247 |     if (object_write(OBJ_BLOB, buf, size, &id) != 0) {
      |         ^~~~~~~~~~~~
index.c:243:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  243 |     fread(buf, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~
index.c: In function 'index_load':
index.c:158:9: warning: '__builtin_strncpy' output may be truncated copying 511 bytes from a string of length 511 [-Wstringop-truncation]
  158 |         strncpy(e->path, path_buf, sizeof(e->path)-1);
      |         ^
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

## 2B:

```
oper@oper-virtualbox:~/PES2UG24CS067-pes-vcs$ xxd .pes/objects/XX/YYYYYYYY... | head -20
xxd: .pes/objects/XX/YYYYYYYY...: No such file or directory
oper@oper-virtualbox:~/PES2UG24CS067-pes-vcs$ find .pes/objects -type f
```

# PHASE 3:

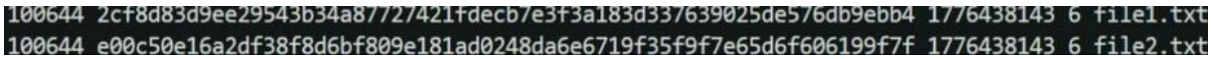
## 3A:

```
oper@oper-virtualbox:~/PES2UG24CS067-pes-vcs$ ./pes status
Staged changes:
  (nothing to show)

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  object.c
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  a.txt
  untracked:  README.md
  untracked:  index.c
  untracked:  tree.h
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  Makefile
```

3B:




PHASE 5:

Q5.1

**How would you implement pes checkout <branch>?**

**Answer:** A branch is just a file in .pes/refs/heads/<branchname> containing a commit hash. To implement checkout:

1. **Read the target branch file** to get the target commit hash.
2. **Read the target commit object** to get its root tree hash.
3. **Recursively expand the tree** into the working directory — for each blob entry write the file, for each sub-tree create the directory and recurse.
4. **Update HEAD** to point to the new branch: write "ref: refs/heads/<branchname>\n" to .pes/HEAD.
5. **Update the index** to reflect the new branch's staged state.

The complexity comes from **detecting and handling dirty working trees** (see Q5.2), removing files that exist on the

current branch but not the target branch, and preserving untracked files that don't conflict.

Q 5.2 **How do you detect a "dirty working directory" conflict?**

**Answer:** Before switching branches, for each file tracked in the *current* index:

1. Check if the file's **mtime or size has changed** compared to its index entry (fast diff, no re-hash).
2. If changed, **re-hash the working file** and compare to the blob hash in the index. If different, the file is dirty.
3. Check if the **target branch's tree has a different blob hash** for the same path.
4. If the working file is dirty AND the target branch changes that file → **refuse checkout** and print an error listing the conflicting files.

This uses only the index (for current staged state) and the object store (for target branch's tree) — no network or external state needed.

Q 5.3 **What is "detached HEAD" and how can commits be recovered?**

**Answer:** Detached HEAD means .pes/HEAD contains a raw commit hash directly (e.g. a1b2c3d4...) instead of a branch reference (e.g. ref: refs/heads/main).

If you make commits in this state, each new commit points to the previous one via its parent field — the commit chain exists in the object store. But when you checkout a branch, HEAD moves to the branch, and there's **no ref pointing to your detached commits**. They become unreachable.

**Recovery:** If you remember the commit hash (e.g. from terminal history or the output of pes commit), you can create a branch pointing to it: create a new file .pes/refs/heads/recovery containing that hash. Then pes checkout recovery brings the commits back.

PHASE 6:

**Q 6.1 Describe a garbage collection algorithm for the object store.**

**Mark-and-Sweep Algorithm:**

1. **Mark phase:** Use a HashSet<ObjectID> to track reachable hashes. Start from every ref (all files in .pes/refs/heads/ and HEAD). For each commit: mark the commit, then mark its tree, recursively mark all tree entries (sub-trees and blobs), and follow the parent pointer to the next commit.
2. **Sweep phase:** Walk all files in .pes/objects/. Any object whose hash is NOT in the reachable set is unreachable — delete it.

**Data structure:** A hash set (unordered set) of 32-byte ObjectIDs is ideal — O(1) lookup per object.

**Estimate for 100,000 commits, 50 branches:** Starting from 50 branches you'd traverse up to 100,000 commits. Each commit has one tree; a typical tree might have 50 entries, so ~5 million tree/blob objects. You'd visit approximately 5–6 million objects in the mark phase. This is the real reason Git's GC is a background operation.

Q 6.2 **Why is concurrent GC with commit dangerous? Describe the race condition.**

**The Race Condition:**

1. GC begins its mark phase. At this moment, refs/heads/main points to commit C2.
2. A concurrent pes add + pes commit starts. It writes new blob objects B3 and tree T3 to the object store — but hasn't updated HEAD yet.
3. GC finishes marking: B3 and T3 are not reachable from any ref, so GC marks them for deletion.
4. GC deletes B3 and T3.
5. The commit operation writes commit C3 (pointing to T3) and updates HEAD to C3.
6. The repository is now **corrupt** — C3 references T3 which no longer exists.

**How Git avoids this:** Git uses a "grace period" — objects written within the last 14 minutes are never deleted by GC, regardless of reachability. This gives any in-progress operations time to complete and create refs pointing to the new objects before GC can remove them.